

Scope of Work

Multilingual Intelligent Document Translation,
Text Reconstruction & Format Preservation System

Python-Based AI Document Processing Application

Author & Creator: Sagnik Sen

sagniklm10@gmail.com • +91 6295862826

“Transforming multilingual documents into accessible English while preserving their original structure, meaning and visual identity.”

Project Type	Python AI/ML Document Processing Application
Author & Creator	Sagnik Sen
Email ID	sagniklm10@gmail.com
Phone Number	+91 6295862826
Primary Function	Extract, reconstruct, translate and regenerate multilingual documents while preserving the original visual structure
Supported Inputs	PDF, Word documents and image-based documents
Target Translation	Indian regional languages to English
Project Duration	7 calendar days from project commencement and receipt of advance
Total Project Cost	INR 5,000
Advance Payment	50% = INR 2,500
Final Payment	50% = INR 2,500 upon completion/delivery

Project Objective

The objective is to develop a Python-based intelligent document-processing application in which a user uploads a PDF, Microsoft Word document, scanned document, or image. The system will automatically extract textual and structural information, identify the source language, reconstruct probable missing or unreadable textual content where technically feasible, translate supported Indian regional-language text into English, and generate a resultant document that preserves the original document’s structure and formatting as closely as possible.

1 Project Scope

The application will implement an end-to-end document intelligence pipeline covering document ingestion, preprocessing, OCR, language detection, contextual reconstruction, translation, layout analysis, format reconstruction, preview and export.

The intended workflow is:

Upload Document → Extract → Understand → Reconstruct → Translate → Rebuild → Preview/Export

1.1 Input Document Support

The first version of the system will target:

- Native text-based PDF documents
- Scanned PDF documents
- Microsoft Word documents
- PNG images
- JPEG/JPG images
- Scanned pages and photographed documents
- Documents containing multilingual Indian-language text

1.2 Language Processing

The architecture will be designed to support Hindi, Punjabi and additional Indian regional languages depending on OCR and translation-model capability.

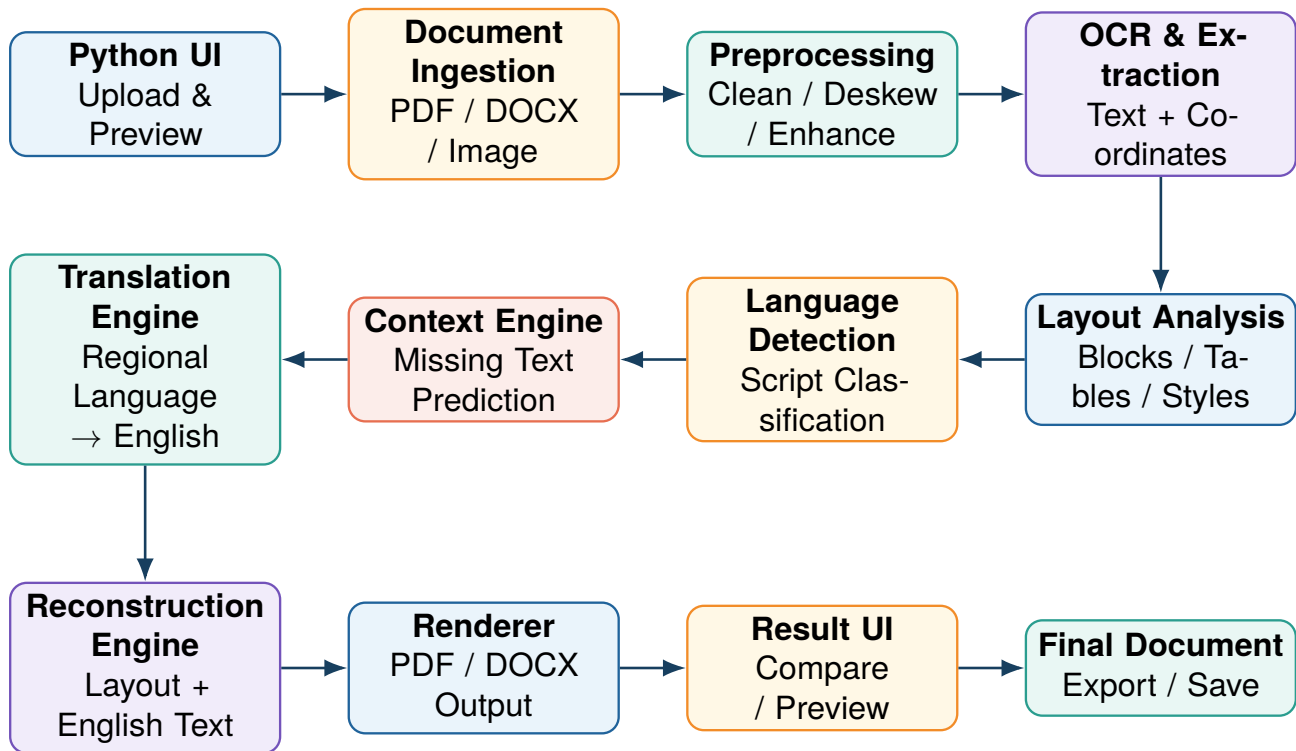
Target language categories include:

Language	Processing Requirement	Output
Hindi	OCR, contextual processing and translation	English
Punjabi	OCR, contextual processing and translation	English
Bengali	Extensible language pipeline	English
Marathi	Extensible language pipeline	English
Gujarati	Extensible language pipeline	English
Tamil	Extensible language pipeline	English
Telugu	Extensible language pipeline	English
Kannada	Extensible language pipeline	English
Malayalam	Extensible language pipeline	English
Odia	Extensible language pipeline	English
Other supported scripts	Subject to OCR/model support	English

2 Functional Requirements

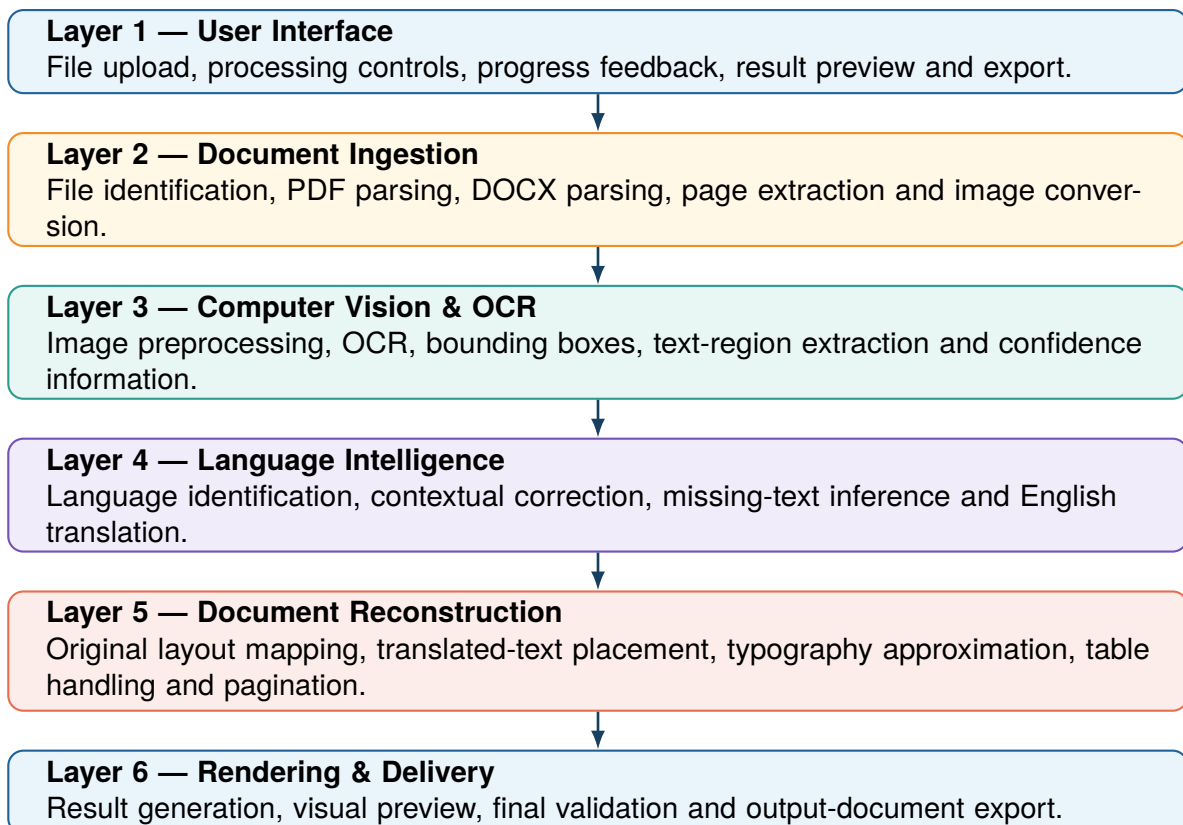
ID	Module	Function
F01	Document Upload	Upload PDF, DOCX, PNG, JPEG and supported document formats.
F02	File Validation	Validate file extension, document readability, page availability and basic processing compatibility.
F03	Document Parsing	Determine whether the document contains native text, scanned images or mixed content.
F04	Image Preprocessing	Perform resizing, denoising, contrast enhancement, thresholding, deskewing and related preprocessing where required.
F05	OCR	Extract textual content and bounding information from scanned pages and images.
F06	Layout Detection	Identify paragraphs, text blocks, headings, tables and other relevant layout regions.
F07	Language Detection	Identify the probable language/script of extracted textual segments.
F08	Missing Text Reconstruction	Use surrounding textual context and language-model reasoning to suggest probable missing or corrupted words where confidence is sufficient.
F09	Translation	Translate extracted regional-language content into English while retaining contextual meaning.
F10	Layout Reconstruction	Place translated English text into corresponding logical regions while approximating the source formatting.
F11	Document Preview	Display the processed result through the Python application interface.
F12	Export	Generate the resultant translated/reconstructed document for user access.

3 Proposed System Architecture



4 Processing Architecture

The processing system will use a modular pipeline so that OCR, translation, reconstruction and UI components can be independently improved or replaced.



5 Internal Data Flow

Each extracted text block will be represented internally with structural metadata similar to:

Document Block Model

Page Number → Block ID → Bounding Coordinates → Source Text → Detected Language → OCR Confidence → Reconstructed Text → English Translation → Formatting Metadata → Output Coordinates

This intermediate representation separates textual intelligence from visual reconstruction and makes the application easier to debug, test and extend.

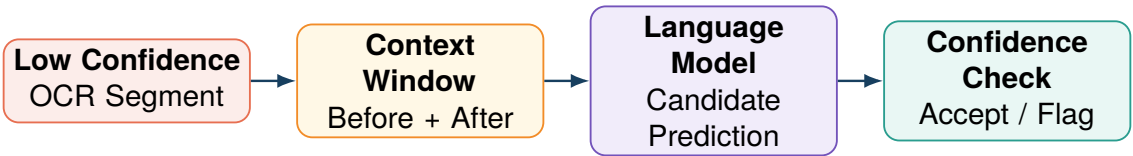
6 Text Extraction Strategy

The application will determine the processing method according to document type.

Input Type	Extraction Method	Processing Strategy
Native PDF	Direct text extraction	Extract text, coordinates, pages and available formatting information.
Scanned PDF	Page rasterization + OCR	Convert pages to images, preprocess and perform OCR.
DOCX	Structured document parsing	Read paragraphs, runs, tables, styles and document structure.
PNG/JPEG	Image OCR	Preprocess image and extract text with spatial information.
Mixed PDF	Hybrid extraction	Use direct extraction for text regions and OCR for image/scanned regions.

7 Missing Word and Pattern Reconstruction

The reconstruction subsystem will attempt to detect uncertain, damaged or missing textual segments using OCR confidence and contextual information.



The model will use neighboring words, sentence semantics, document context and language information to determine probable candidates.

Accuracy Constraint

Missing-text reconstruction is probabilistic. The system will therefore treat predicted content separately from directly recognized content. Where confidence is insufficient, the pipeline should preserve or flag the uncertain segment instead of representing an uncertain prediction as guaranteed source text.

8 Translation Pipeline

Translation will be performed after source text has been normalized and reconstructed.

Step	Process	Purpose
1	Source Language Detection	Identify language/script for each relevant text segment.
2	Text Normalization	Normalize extracted Unicode text and remove OCR artefacts where appropriate.
3	Context Reconstruction	Resolve probable incomplete words or corrupted sequences.
4	Translation	Translate the normalized source text into English.
5	Context Validation	Check basic contextual consistency of the translated segment.
6	Layout Mapping	Associate English translation with the source text block and coordinates.
7	Rendering	Insert translated text into the reconstructed output document.

9 Format Preservation Strategy

Exact pixel-level reproduction cannot be guaranteed for every arbitrary input because translated English text can differ significantly in character length, line breaking, font metrics and page geometry from the original language.

The system will therefore target **structural and visual preservation** through:

- Original page dimensions
- Relative text-block positions
- Paragraph grouping
- Heading hierarchy
- Text alignment
- Approximate font sizing
- Bold/italic information where extractable
- Tables and cell relationships where technically detectable
- Images and non-text graphical regions where feasible
- Margins and spacing
- Page ordering
- Section relationships

Reflow Strategy

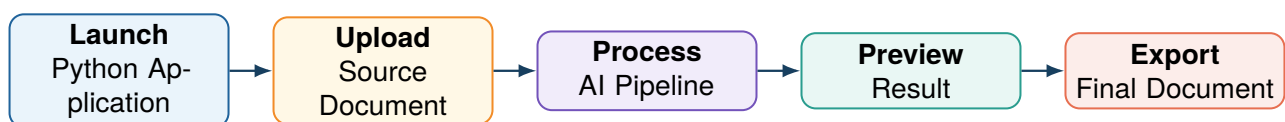
If translated English text exceeds the original bounding region, the reconstruction engine will use controlled font adjustment, wrapping, spacing and block expansion where possible. The objective is to maximize readability while remaining visually close to the source document.

10 Proposed Python Technology Stack

Component	Candidate Technology	Purpose
Programming Language	Python	Core application and orchestration
User Interface	Streamlit / Gradio	Upload, processing, preview and export UI
PDF Processing	PyMuPDF / pdfplumber	PDF parsing, page geometry and text extraction
Word Processing	python-docx	DOCX reading and output reconstruction
Image Processing	OpenCV / Pillow	Preprocessing and image manipulation
OCR Layer	Tesseract / EasyOCR / compatible multilingual OCR	Regional-language text extraction
Language Detection	Script/language classifier	Identification of source language
Translation	Indic translation model / multilingual transformer	Regional Indian language to English translation
Context Prediction	Transformer-based language model	Contextual restoration of uncertain or missing words
Document Rendering	PyMuPDF / ReportLab / python-docx	Generate final translated output
Temporary Processing	Python filesystem / temporary workspace	Intermediate pages, images and extracted metadata

The final implementation may substitute equivalent libraries or models where this improves compatibility, language coverage, deployment feasibility or output quality.

11 Application UI Workflow



The user experience will follow a simple execution model:

1. Run the Python application.
2. Application UI opens locally or through the configured interface.
3. User selects or uploads the source document.
4. System identifies the document type.
5. Text and layout information are extracted.
6. Regional-language text is identified.
7. Low-confidence or incomplete text is evaluated.
8. Source text is translated into English.
9. Document structure is reconstructed.

10. Resultant document is rendered.
11. User views the processed document.
12. User exports the final result.

12 Software Module Structure

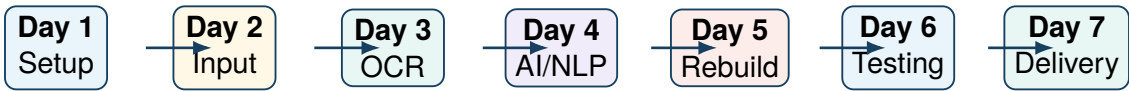
A modular source-code structure is proposed as follows:

```
document_translator/  
  |- app.py  
  |- config/  
    |- settings.py  
  |- core/  
    |- document_loader.py  
    |- pdf_processor.py  
    |- docx_processor.py  
    |- image_processor.py  
    |- ocr_engine.py  
    |- layout_engine.py  
    |- language_detector.py  
    |- context_engine.py  
    |- translation_engine.py  
    |- reconstruction_engine.py  
    |- renderer.py  
  |- models/  
    |- model_manager.py  
  |- ui/  
    |- components.py  
  |- utils/  
    |- file_utils.py  
    |- text_utils.py  
    |- validation.py  
  |- outputs/  
  |- requirements.txt  
  |- README.md
```


13 Seven-Day Development Roadmap

Day	Phase	Planned Activities	Output
1	Architecture & Setup	Project structure, dependency setup, processing architecture, file validation and basic UI skeleton.	Base Application
2	Document Ingestion	PDF, DOCX and image loaders; document classification; page extraction and preprocessing pipeline.	Input Pipeline
3	OCR & Layout	Multilingual OCR integration, coordinate extraction, block segmentation and layout metadata.	Extraction Engine
4	Language Intelligence	Language detection, text normalization, missing-word/context prediction and translation integration.	AI/NLP Pipeline
5	Reconstruction	Map translated content back to document regions, preserve structure, implement output rendering.	Output Engine
6	UI Integration & Testing	Connect upload, processing, progress, preview and export flows; test representative multilingual documents.	Integrated Build
7	Validation & Delivery	Bug fixing, workflow validation, documentation, dependency finalization and delivery package preparation.	Final Release

14 Roadmap Visualization



15 Deliverables

The project delivery will include:

No.	Deliverable	Status
01	Python source code for the document-processing application	Included
02	User interface for document upload and processing	Included
03	PDF/document/image ingestion pipeline	Included
04	OCR processing module	Included
05	Regional-language identification module	Included
06	Missing-text/context prediction module	Included
07	Regional-language to English translation module	Included
08	Document layout reconstruction module	Included
09	Result preview/export workflow	Included
10	Python dependency specification	Included
11	Basic installation and execution documentation	Included
12	Final integrated prototype	Included

16 Acceptance Criteria

The solution will be considered functionally delivered when the application can demonstrate the following end-to-end workflow on mutually agreed representative test documents:

- Application starts successfully in the agreed Python environment.
- User can upload a supported PDF, DOCX or image file.
- System can extract text from a supported representative document.
- OCR is applied to scanned/image-based content where required.
- Source language is detected or correctly routed through the language pipeline.
- Supported Indian-language text can be translated into English.
- Contextual reconstruction can generate candidate text for suitable incomplete/low-confidence regions.
- Output maintains the major structural characteristics of the source document.
- Processed result can be previewed through the application.
- Resultant document can be generated/exported successfully.

17 Quality and Validation Strategy

Testing will be divided into four categories.

Test Category	Validation
Input Testing	PDF, DOCX, PNG and JPEG handling; corrupted/unsupported-file behavior.
OCR Testing	Readable scans, low-resolution documents, multilingual text and OCR confidence behavior.
NLP Testing	Language identification, contextual restoration and translation consistency.
Output Testing	Page structure, text positioning, wrapping, tables, output readability and export operation.

18 Error Handling

The application will implement controlled handling for common failure scenarios, including:

- Unsupported file type
- Empty document
- Corrupted document
- OCR failure
- Unsupported or uncertain language
- Translation-model failure
- Very low OCR confidence
- Invalid page structure
- Output rendering failure
- Temporary file-processing errors

Where automatic recovery is not appropriate, the UI will return a meaningful processing status instead of silently generating unreliable output.

19 Important Technical Constraints

OCR Accuracy

OCR accuracy depends on scan resolution, image quality, font type, handwriting, background noise, document orientation and language/model support. Perfect recognition of arbitrary documents is not guaranteed.

Translation Accuracy

Machine translation can preserve meaning effectively for many documents but cannot be guaranteed to reproduce every linguistic, legal, technical or contextual nuance of a professional human translation.

Missing Content Prediction

Content that is physically absent from the source cannot be recovered with certainty. The application can infer probable words or patterns from context, but such output constitutes a model prediction rather than recovered original information.

Format Fidelity

The target is high structural similarity rather than guaranteed pixel-identical reproduction. Complex typography, handwriting, unusual fonts, irregular tables, mathematical notation, decorative designs and highly graphical documents may require additional handling.

20 Project Boundaries

The INR 5,000 scope covers development of the described prototype/MVP within the seven-day delivery period.

Unless separately agreed, the following are outside the base project scope:

- Enterprise-scale cloud infrastructure
- Paid third-party API charges

- GPU/cloud hosting charges
- Commercial OCR licensing charges
- Commercial translation API charges
- Large-scale model training from scratch
- Creation of proprietary multilingual training datasets
- Handwritten-document specialization
- Mobile application development
- Production-grade multi-user authentication
- Enterprise database infrastructure
- High-volume batch-processing infrastructure
- Guaranteed pixel-perfect reconstruction for every arbitrary document
- Human-certified translation
- Human-certified recovery of missing source text

21 Commercial Proposal

Formal Quotation

Prepared by: Sagnik Sen, Author & Creator

Email: sagniklm10@gmail.com **Phone:** +91 6295862826

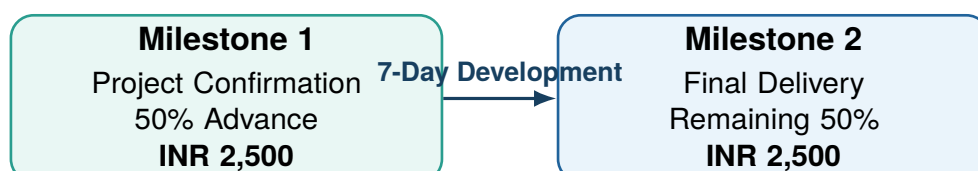
Quoted project value: INR 5,000

Payment terms: 50% advance and 50% upon completion/delivery.

Estimated delivery: 7 calendar days from project commencement, receipt of advance, and availability of the required sample documents.

Commercial Item	Amount / Term
Total Project Cost	INR 5,000
Advance Payment	INR 2,500
Advance Percentage	50%
Balance Payment	INR 2,500
Balance Percentage	50%
Development Period	7 Calendar Days
Project Start	After advance payment and availability of required sample/input documents
Final Delivery	Upon completion of the agreed seven-day development cycle

22 Payment Milestones



23 Project Assumptions

The project schedule assumes:

- Representative sample documents are provided at project commencement.
- Required development dependencies can be installed in the target environment.
- The selected OCR/translation models support the required scripts sufficiently for prototype implementation.
- Any required paid API credentials, if explicitly selected, are provided by the client.
- Scope remains consistent with this document during the seven-day implementation period.
- Major new features requested after commencement may require revised effort, schedule and commercial terms.

24 Security and Document Handling

The application architecture will prefer temporary/local document processing where the selected models and libraries permit it.

If an external AI, OCR or translation API is selected during implementation, document content transmitted to that service will be subject to the respective provider's processing and privacy terms.

Sensitive or confidential documents should therefore be processed using an approved deployment configuration appropriate to the confidentiality requirements of the intended environment.

25 Future Expansion Architecture

The modular architecture allows subsequent extension toward:

- Additional Indian languages
- Batch document processing
- REST API integration
- GPU inference
- Cloud deployment
- User authentication
- Document history
- Side-by-side source/translation comparison
- Confidence highlighting
- Human correction interface
- Table-specific translation
- Custom terminology dictionaries
- Domain-specific translation models
- Document classification
- Searchable translated document repository

26 Success Definition

Source Document → Automated Extraction → Contextual Reconstruction → English Translation → Layout Preservation → Resultant Document

The successful outcome of this project is a functional Python prototype that provides a unified interface for uploading a supported multilingual document and obtaining a translated English document whose structure and visual organization remain as close as technically practical to the original.

27 Final Project Summary

Solution	AI-Assisted Multilingual Document Translation and Reconstruction System
Author & Creator	Sagnik Sen
Contact	sagniklm10@gmail.com • +91 6295862826
Implementation	Python-based modular application
Input	PDF, Word and image documents
Core Intelligence	OCR + language detection + contextual reconstruction + translation + layout reconstruction
Source Languages	Hindi, Punjabi and extensible support for other Indian regional languages
Target Language	English
User Experience	Run application → upload document → process → preview → export
Output Goal	English resultant document preserving the source structure and formatting as closely as technically feasible
Development Period	7 Calendar Days
Project Value	INR 5,000
Advance	50% — INR 2,500
Balance	50% — INR 2,500

“Technology is most valuable when it makes information clearer, more accessible and easier to preserve.”